

Robotics Project – Code Documentation

7 June 2025

Contents

1	compute_path.cpp	2
1.1	Purpose & Context	2
1.2	ROS Interfaces	2
1.3	Key Helpers / Algorithms	2
1.4	Step-by-Step Flow	2
2	compute_trajectory.cpp	2
2.1	Purpose & Context	2
2.2	ROS Interfaces	3
2.3	Key Helpers / Algorithms	3
2.4	Step-by-Step Flow	3
3	kinematics.cpp	3
3.1	Purpose & Context	3
3.2	ROS Interfaces	3
3.3	Key Helpers / Algorithms	3
3.4	Step-by-Step Flow	4
4	table_avoidance.cpp	4
4.1	Purpose & Context	4
4.2	ROS Interfaces	4
4.3	Key Helpers / Algorithms	4
4.4	Step-by-Step Flow	4
5	control_unit.cpp	4
5.1	Purpose & Context	4
5.2	ROS Interfaces	5
5.3	Key Helpers / Algorithms	5
5.4	Step-by-Step Flow	5
6	trajectory_execution.cpp	5
6.1	Purpose & Context	5
6.2	ROS Interfaces	5
6.3	Key Helpers / Algorithms	5
6.4	Step-by-Step Flow	5
7	gripper_service.cpp	5
7.1	Purpose & Context	5
7.2	ROS Interfaces	6
7.3	Key Helpers / Algorithms	6
7.4	Step-by-Step Flow	6

8	publish_trajectory_node.cpp	6
8.1	Purpose & Context	6
8.2	ROS Interfaces	6
8.3	Key Helpers / Algorithms	6
8.4	Step-by-Step Flow	6
9	detection.cpp	6
9.1	Purpose & Context	6
9.2	ROS Interfaces	6
9.3	Key Helpers / Algorithms	7
9.4	Step-by-Step Flow	7
10	position.cpp	7
10.1	Purpose & Context	7
10.2	ROS Interfaces	7
10.3	Key Helpers / Algorithms	7
10.4	Step-by-Step Flow	7

1 compute_path.cpp

1.1 Purpose & Context

Node that converts a *start/end* pose pair into a discretised Cartesian path. It lives in the `motion` package and publishes an interpolated pose array that later becomes a joint-space trajectory.

1.2 ROS Interfaces

- **Publisher** `/computed_trajectory` (`geometry_msgs/PoseArray`, transient-local)
- **Publisher** `/computed_path` (`geometry_msgs/PoseArray`)
- **Subscriber** `/path_extrema` (`custom_msg_interfaces/StartEndPosition`)

1.3 Key Helpers / Algorithms

- `compute_interpolated_poses()` – samples N evenly-spaced interpolation factors between 0 and 1.
- `s_lerp()` – quaternion SLERP + linear translation ($SE(3)$ interpolator).

1.4 Step-by-Step Flow

1. Constructor sets up QoS, publishers, and subscriber.
2. Callback `handle_compute_path()` fires on start/end message and calls `compute_interpolated_poses()`.
3. For each sample t compute pose via `s_lerp()`.
4. Aggregate into a `PoseArray`, log each waypoint, then publish.

2 compute_trajectory.cpp

2.1 Purpose & Context

Implements an on-demand joint-trajectory generator that transforms a *waypoint pose array* into time-parameterised joint commands for the UR5. It is written as an `rclcpp` node called **ComputeTrajectoryService**.

2.2 ROS Interfaces

- **Subscriber** `joint_states` (`sensor_msgs/JointState`) – current joint angles.
- **Subscriber** `/computed_path` (`geometry_msgs/PoseArray`) – Cartesian waypoints.
- **Publisher** `/computed_trajectory` (`custom_msg_interfaces/ViaPoints`) – joint positions with timing.
- **(optional)** Service client for inverse kinematics.

2.3 Key Helpers / Algorithms

- `compute_cubic_coefficients()` – analytical ($a_0 \dots a_3$) solver for a single joint segment.
- `generate_cubic_trajectory()` – wraps coefficient generation for all 6 joints.
- `select_closest_one()` – chooses IK solution with minimal Δ -norm to previous set.
- `ur5_singAvoid()` / `computeChainFK()` – checks Jacobian determinant to avoid singularities.

2.4 Step-by-Step Flow

1. Wait until current `joint_states` arrive.
2. Upon new `PoseArray`: run IK for each waypoint $\rightarrow$ matrix of candidate joint sets.
3. For every segment compute boundary velocities v_0, v_1 via finite difference.
4. Solve cubic coefficients per joint, build a `JointTrajectory`.
5. Check singularities; rescale or reject if necessary.
6. Publish the finished trajectory and log debug information.

3 kinematics.cpp

3.1 Purpose & Context

The **KinematicsNode** hosts both forward and inverse kinematics as ROS2 services for the UR5: it exposes a single node (`kinematics_node`) with two services rather than separate server classes.

3.2 ROS Interfaces

- **Service** `/compute_dir_kin` (`custom_msg_interfaces/srv/ComputeDirKin`) – given a 6-vector of joint angles, returns end-effector pose.
- **Service** `/compute_ik` (`custom_msg_interfaces/srv/ComputeIK`) – given a desired end-effector pose, returns up to 8 joint-angle solutions.

3.3 Key Helpers / Algorithms

- `ur5Direct()` – DH-parameter chain evaluation. Builds $T_{0\dots6}$, outputs (p, R) .
- `ur5Inverse()` – algebraic inverse kinematics, computing up to 8 solutions.

3.4 Step-by-Step Flow

1. **Constructor** registers two services on startup:
 - `compute_dir_kin` calls `computeDirectKinematics()`.
 - `compute_ik` calls `computeInverseKinematics()`.
2. *computeDirectKinematics*:
 - (a) Read incoming joint vector.
 - (b) Call `ur5Direct()` to build pose and intermediate transforms.
 - (c) Fill response pose fields and status.
3. *computeInverseKinematics*:
 - (a) Extract translation and quaternion from request.
 - (b) Convert to Eigen **R** and **p**.
 - (c) Call `ur5Inverse()` to get the 8×6 solution matrix.
 - (d) Pack solutions into `Float64MultiArray` layout and return.
4. Both paths log success or error and return a status message.

4 table_avoidance.cpp

4.1 Purpose & Context

Utility node **EndEffectorHeightChecker**. Watches TF from `base_link` to `tool0` and warns (or shuts down) if the tool drops below table height.

4.2 ROS Interfaces

- **TF listener** via `tf2_ros::Buffer/TransformListener`
- **Publisher** `/shutdown` (`std_msgs/Bool`)

4.3 Key Helpers / Algorithms

Single timer callback `checkEndEffectorHeight()` every 100 ms.

4.4 Step-by-Step Flow

1. Look up transform once per timer iteration.
2. Read z translation; if $< z_{\text{threshold}}$ publish shutdown.

5 control_unit.cpp

5.1 Purpose & Context

High-level finite-state machine **ControlNode** that coordinates *perception* $\rightarrow$ *planning* $\rightarrow$ *execution*.

5.2 ROS Interfaces

- Subscriber `/inference_3d` (`custom_msg_interfaces/ClassPose`)
- Subscriber `/trajectory_executed` (`std_msgs/Bool`)
- Publisher `/path_extrema` (`custom_msg_interfaces/StartEndPosition`)
- Publisher `/execute_trajectory` (`std_srvs/Trigger`)
- TF listener `camera` $\rightarrow$ base transform

5.3 Key Helpers / Algorithms

`move_camera_frame_to_world()` helper in perception callback; median filter over recent detections.

5.4 Step-by-Step Flow

1. Project detected object pose into world frame.
2. Compose safe approach pose, publish `/path_extrema`.
3. Wait for `/trajectory_executed`, then proceed.

6 `trajectory_execution.cpp`

6.1 Purpose & Context

Action client node forwarding joint trajectories to the MoveIt `scaled_joint_trajectory_controller` and monitoring result state.

6.2 ROS Interfaces

- Subscriber `/computed_trajectory` (`custom_msg_interfaces/ViaPoints`)
- Action client `/scaled_joint_trajectory_controller/follow_joint_trajectory`
- Publisher `/trajectory_executed` (`std_msgs/Bool`)

6.3 Key Helpers / Algorithms

Lambda `send_goal()` inside constructor.

6.4 Step-by-Step Flow

1. Convert `ViaPoints` to `JointTrajectory` goal message.
2. Send goal asynchronously; register feedback and result callbacks.
3. Publish `/trajectory_executed` based on result code.

7 `gripper_service.cpp`

7.1 Purpose & Context

Wraps the Robotiq-style gripper into simple ROS2 Trigger services. Executes URScript files over a TCP socket to open/close or go neutral.

7.2 ROS Interfaces

- Service `/open_gripper` (`std_srvs/Trigger`)
- Service `/close_gripper` (`std_srvs/Trigger`)
- Service `/neutral_gripper` (`std_srvs/Trigger`)
- Socket port 30002 to robot controller

7.3 Key Helpers / Algorithms

`send_script()` – pushes a `.script` file line-by-line over TCP.

7.4 Step-by-Step Flow

1. On Trigger request send corresponding script; return success status.
2. Maintains `current_state_` string for quick introspection.

8 `publish_trajectory_node.cpp`

8.1 Purpose & Context

Standalone action-client demo node publishing hard-coded trajectories followed by gripper operations, useful for calibration and offline tests.

8.2 ROS Interfaces

- Action client `/scaled_joint_trajectory_controller/follow_joint_trajectory`
- Service clients `/open_gripper`, `/close_gripper`
- Subscriber `joint_states` (`sensor_msgs/JointState`)

8.3 Key Helpers / Algorithms

`prepare_trajectories()` constructs a vector of `JointTrajectory` messages; `send_next_trajectory()` chains them.

8.4 Step-by-Step Flow

1. Wait for action server and services at startup.
2. Queue trajectories; on success callback, chain next trajectory or gripper command.

9 `detection.cpp`

9.1 Purpose & Context

Runs a TorchScript-based object detector on RGB frames and publishes bounding boxes and class IDs.

9.2 ROS Interfaces

- Subscriber `/camera/color/image_raw` (`sensor_msgs/Image`)
- Publisher `/detections_2d` (`std_msgs/Float32MultiArray`)
- Parameters `model_path`, `confidence_threshold`, `device`

9.3 Key Helpers / Algorithms

`preprocess()` – resize, normalise, convert to tensor; `postprocess()` – NMS and conversion.

9.4 Step-by-Step Flow

1. Convert ROS image to `cv::Mat` via `cv_bridge`.
2. Run detector forward pass; fall back to CPU if necessary.
3. Publish detections; overlay debug image if enabled.

10 position.cpp

10.1 Purpose & Context

Triangulates 3-D position of detected objects by fusing 2-D bounding box and depth point cloud.

10.2 ROS Interfaces

- **Subscriber** `/inference_2d` (`std_msgs/Float32MultiArray`)
- **Subscriber** `/camera/depth/points` (`sensor_msgs/PointCloud2`)
- **Publisher** `/inference_3d` (`custom_msg_interfaces/ClassPose`)
- **TF listener** `camera` → world transform

10.3 Key Helpers / Algorithms

`calc_centroid()` – average XYZ within bounding box; `publish_pose()` – transforms centroid into base frame.

10.4 Step-by-Step Flow

1. For each 2-D detection correlate to depth cloud and compute centroid.
2. Transform centroid with `tf2`, wrap in `ClassPose`, publish.
3. Handles missing depth gracefully (NaN guard + fallback).